

**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



BLM4522 - AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ

Final Teslim Raporu

**Bora KOCABIYIK
21290270**

Öğr. Gör. Enver BAĞCI

10/06/2026

github.com/brckfrc/database-operations-lab

ÖZET

Bu rapor, Ankara Üniversitesi Bilgisayar Mühendisliği Bölümü BLM4522 “Ağ Tabanlı Paralel Dağıtım Sistemleri” dersi kapsamında yürütülen veritabanı operasyonları çalışmalarının final dönemi teslimini kapsamaktadır. Çalışmanın bütünü, beş bağımsız veritabanı projesinden oluşan ve her biri ayrı bir veritabanı yönetim sistemi ile gerçek bir senaryo üzerinde kurulan bir laboratuvar deposu (database-operations-lab) etrafında düzenlenmiştir. Bu projelerden ikisi (performans optimizasyonu ve veri temizleme/ETL süreçleri) vize döneminde teslim edilmiş olup, elinizdeki final raporu kalan üç projeyi ele almaktadır. Üç proje de yalnızca kuramsal anlatımla değil, çalışan sistemler üzerinde uygulanarak ve elde edilen sonuçlar ekran görüntüleriyle belgelenerek sunulmuştur.

Projelerin ortak yaklaşımı, her senaryoda önce sorunlu ya da eksik bir başlangıç durumunu ortaya koymak, ardından uygulanan çözümün etkisini önce-sonra karşılaştırmasıyla somut biçimde kanıtlamaktır. Bu yaklaşım, yapılan işin yalnızca tarif edilmesini değil, gerçekten çalıştığının gösterilmesini amaçlar. Kullanılan veritabanları ve araçlar, ele alınan konunun gerektirdiği gerçek dünya standartlarına göre seçilmiştir.

Raporda sırasıyla şu üç proje sunulmaktadır. Proje 2, PostgreSQL üzerinde sürekli arşivleme (Write-Ahead Logging) temelli bir yedekleme altyapısı kurarak, yanlışlıkla veri silme ve tablo düşürme gibi felaket senaryolarında veritabanını istenen bir ana geri döndüren zamana dayalı kurtarma (Point-In-Time Recovery) yeteneğini ele alır. Proje 3, Oracle üzerinde bir hastane bilgi sistemi senaryosuyla; şifreleme, veri maskeleyme, rol tabanlı erişim kontrolü, SQL enjeksiyonuna karşı savunma ve denetim kayıtlarını içeren katmanlı bir güvenlik mimarisi kurar. Proje 4 ise birbirinden bağımsız üç gerçek sunucu üzerinde, şifreli bir özel ağ ile haberleşen; sunucu arızasında otomatik devralma (failover) ve okuma/yazma yük dengeleme yeteneğine sahip, dağıtık ve yüksek erişilebilir bir PostgreSQL kümesi tasarlar.

Her proje; amacı, kullanılan ortam ve araçları, başlangıç durumu, uygulanan işlemler, elde edilen sonuçlar ve karşılaşılan problemlerle çözümleri başlıkları altında, ekran görüntüleriyle desteklenmiş biçimde anlatılmaktadır. Projelerin tamamına ait kaynak kodları, yapılandırma dosyaları, betikler ve ek belgeler, kapakta belirtilen genel ağ (GitHub) deposunda açık biçimde yer almaktadır.

İÇİNDEKİLER

PROJE 2: VERİTABANI YEDEKLEME VE FELAKETTEN KURTARMA	5
1. Projenin Amacı	5
1.1. Giderilmesi Hedeflenen Problemler	5
2. Kullanılan Ortam ve Araçlar	5
3. Kullanılan Veri Seti ve Veritabanı	5
4. Başlangıç Durumu — pg_dump Neden Yetersiz?	6
5. Yedekleme Altyapısının Kurulması	6
5.1. Ortam ve Yapılandırma	6
5.2. Yedekleme Stratejisi	6
6. Felaket Senaryoları ve Zamana Dayalı Kurtarma (PITR)	7
6.1. Felaket A — Yanlışlıkla Toplu Silme	7
6.2. Felaket B — Kritik Tablonun Düşürülmesi	7
7. Yedeklerin Doğrulanması	8
8. Elde Edilen Sonuçlar	9
9. Karşılaşılan Problemler ve Çözümleri	9
9.1. Salt-Okunur Yedek Deposu Hatası	9
9.2. Yanlış Hedefe Geri Yükleme	9
9.3. Kurulum Sırasında Geçici Kesinti	10
10. Sonuç ve Değerlendirme	10
PROJE 3: VERİTABANI GÜVENLİĞİ VE ERIŞİM KONTROLÜ	11
1. Projenin Amacı	11
2. Kullanılan Ortam ve Araçlar	11
3. Kullanılan Veri Seti ve Veritabanı	11
4. Başlangıç Durumu — Güvensiz Sistem	11
5. Uygulanan Güvenlik Denetimleri	13
5.1. Rol Tabanlı Erişim Kontrolü	13
5.2. Hassas Alanın AES-256 ile Şifrelenmesi	13
5.3. Yetkiye Göre Veri Maskeleyme	14
5.4. SQL Enjeksiyonuna Karşı Savunma	15
5.5. Denetim (Audit) Kayıtları ve Görev Ayrılığı	16
6. Elde Edilen Sonuçlar	17
7. Karşılaşılan Problemler ve Çözümleri	18
7.1. Şifreleme Yetkisinin Verilmesi	18
7.2. Test Bağlantılarının Güvenilirliği	18
7.3. Sürüm Kaynaklı Uyarlamalar	18
8. Sonuç ve Değerlendirme	18
PROJE 4: VERİTABANI YÜK Dengeleme ve Dağıtık Yapılar	19
1. Projenin Amacı	19
1.1. Giderilmesi Hedeflenen Problemler	19
2. Kullanılan Ortam ve Araçlar	19
2.1. Çok Mimarili (Heterojen) Donanım	20
3. Sistem Mimarisi ve Ağ Altyapısı	20
3.1. WireGuard Şifreli Özel Ağ	20
3.2. etcd Üç Döğümlü Konsensüs Kümesi	21
3.3. Neden Üç Döğüm? — Çoğunluk (Quorum) Gereksinimi	21
3.4. Kurulum Adımları	21

4. Akışlı Çoğaltma (Streaming Replication)	21
5. Otomatik Devralma (Failover)	22
5.1. Devralma Senaryosu	22
5.2. Geri Katılım (Failback)	22
6. Yük Dengeleme ve Yük Dengeleyici Yedekliliği	23
6.1. Okuma/Yazma Ayrımı	23
6.2. Yük Dengeleyici Katmanının Yedeklenmesi	23
7. Elde Edilen Sonuçlar	24
8. Karşılaşılan Problemler ve Çözümleri	24
8.1. Güvenlik Duvarının Ağ Trafikini Engellemesi	24
8.2. etcd Küme Sürümünün Yanlış Raporlanması	24
8.3. Konteyner ve Ağ Kaynaklı Diğer Sorunlar	24
9. Sonuç ve Değerlendirme	25
KAYNAKLAR	26

PROJE 2: VERİTABANI YEDEKLEME VE FELAKETTEN KURTARMA

1. Projenin Amacı

Bu projenin temel amacı, geleneksel veritabanı yedekleme yöntemlerinin (özellikle yalnızca anlık bir kopya alan pg_dump aracının) büyük ve sürekli işlem gören veritabanlarında neden yetersiz kaldığını göstermek ve bunun yerine endüstri standardı olan, sürekli arşivleme (Write-Ahead Logging, WAL) temelli bir yedekleme altyapısı kurmaktır. Proje kapsamında üç farklı yedek türü (tam, diferansiyel ve artımlı) alınmış; ardından gerçekçi veri kaybı senaryoları (yanlışlıkla veri silme ve tablo düşürme) tetiklenerek, veritabanının istenen bir saniyeye geri döndürülmesi (Point-In-Time Recovery, PITR) süreci uçtan uca gerçekleştirilmiştir.

1.1. Giderilmesi Hedeflenen Problemler

- Yanlışlıkla veri silinmesi (kapsamı geniş, koşulsuz silme sorguları).
- Kritik bir tablonun yapısıyla birlikte düşürülmesi.
- Yalnızca günlük anlık kopya almanın, iki kopya arasındaki işlemleri kaybettirmesi.

2. Kullanılan Ortam ve Araçlar

Sistem, üretim ortamlarında yaygın olarak kullanılan profesyonel araçlarla kurulmuştur. Bileşenler ve görevleri aşağıda özetlenmiştir.

- PostgreSQL 16 — Veri katmanı; tüm yedekleme ve kurtarma işlemlerinin uygulandığı veritabanı.
- pgBackRest — PostgreSQL için geliştirilmiş, tam/diferansiyel/artımlı yedeklemeyi ve zamana dayalı kurtarmayı doğal olarak destekleyen profesyonel yedekleme aracı.
- Docker ve Docker Compose — Asıl veritabanı ile yedek doğrulama ortamının izole biçimde ayağa kaldırılması.
- Cron — Yedekleme işlerinin belirli aralıklarla otomatik çalıştırılması (zamanlanmış görevler).

3. Kullanılan Veri Seti ve Veritabanı

Senaryo, gerçekçi bir kriz hissi yaratması için örnek bir bankacılık veritabanı (bankdb) etrafında kurgulanmıştır. Banka işlemleri gibi kritik verilerin söz konusu olması, “yanlışlıkla silme” veya “tablo düşürme” gibi felaketleri daha anlamlı kılar.

Çizelge 1. bankdb şeması ve veri hacmi

Tablo	İçerik	Hacim
customers	Müşteri bilgileri (ad, e-posta, kayıt tarihi)	—
accounts	Banka hesapları (IBAN, bakiye, para birimi)	—
transactions	Hesaplar arası para hareketleri	≈ 505.000 satır

transactions tablosundaki yaklaşık 505 bin satırlık sentetik veri, yedekler arasındaki boyut farklarının ve kurtarma süresinin anlamlı biçimde gözlemlenebilmesi için bilinçli olarak yüksek tutulmuştur.

4. Başlangıç Durumu — pg_dump Neden Yetersiz?

Proje, herhangi bir kurtarma planının bulunmadığı, yalnızca standart PostgreSQL imajının kullanıldığı bir başlangıç durumuyla açılır. Bu aşamada geleneksel pg_dump aracıyla bir anlık kopya alınmış; ardından bu kopyanın, alındığı andan sonra gerçekleşen işlemleri kapsamadığı ve dolayısıyla iki yedekleme arasındaki bir felakette veri kaybına yol açtığı kanıtlanmıştır. Bu “önce” durumu, sonraki bölümlerde kurulan WAL temelli sürekli arşivleme altyapısının neden gerekli olduğunun somut gerekçesini oluşturur.

5. Yedekleme Altyapısının Kurulması

5.1. Ortam ve Yapılandırma

İki konteyner ayağa kaldırılmıştır: asıl veritabanını barındıran birincil konteyner ve yedeklerin sağlamlığını test etmek için kullanılan kullan-at (ephemeral) bir doğrulama konteyneri. pgBackRest yapılandırmasında yedek tutma politikası (retention) belirlenmiş (örneğin en az iki tam ve dört diferansiyel yedek saklanacak biçimde); PostgreSQL yapılandırması ise WAL arşivlemeyi etkinleştirecek şekilde düzenlenmiştir. Yedek alma işleri, üretim gerçekliğini yansıtmaları için cron ile belirli aralıklarla otomatik çalışacak biçimde zamanlanmıştır.

5.2. Yedekleme Stratejisi

pgBackRest üç katmanlı bir yedekleme stratejisini destekler. Bu üç türün birlikte kullanımı, hem depolama alanından tasarruf sağlar hem de hızlı kurtarmaya olanak tanır.

Çizelge 2. Yedekleme türleri ve özellikleri

Yedek Türü	Kapsam	Avantaj
Tam (Full)	Veritabanının eksiksiz kopyası	Kurtarmanın temelini oluşturur
Diferansiyel (Diff)	Son tam yedekten beri değişenler	Hızlı kurtarma
Artımlı (Incremental)	Son herhangi bir yedekten beri değişenler	En az yer kullanımı

```

bora@bora-MacBook-Air project-2-backup-recovery % ./scripts/06_show_backup_info.sh

Showing pgBackRest repository info...
stanza: bankdb
  status: ok
  cipher: none

db (current)
  wal archive min/max (16): 000000010000000000000001/000000040000000000000020

  full backup: 20260528-104248F
    timestamp start/stop: 2026-05-28 10:42:48+00 / 2026-05-28 10:42:56+00
    wal start/stop: 00000001000000000000000A / 00000001000000000000000A
    database size: 31.7MB, database backup size: 31.7MB
    repo1: backup set size: 4.3MB, backup size: 4.3MB

  diff backup: 20260528-104248F_20260528-104358D
    timestamp start/stop: 2026-05-28 10:43:58+00 / 2026-05-28 10:48:30+00
    wal start/stop: 000000010000000000000011 / 000000010000000000000011
    database size: 75.1MB, database backup size: 47.4MB
    repo1: backup set size: 17.2MB, backup size: 13.6MB
    backup reference total: 1 full

  incr backup: 20260528-104248F_20260528-104852I
    timestamp start/stop: 2026-05-28 10:48:52+00 / 2026-05-28 10:49:11+00
    wal start/stop: 000000010000000000000013 / 000000010000000000000013
    database size: 75.5MB, database backup size: 43.7MB
    repo1: backup set size: 17.3MB, backup size: 13.1MB
    backup reference total: 1 full, 1 diff

  incr backup: 20260528-104248F_20260529-020001I
    timestamp start/stop: 2026-05-29 02:00:01+00 / 2026-05-29 02:00:08+00
    wal start/stop: 000000030000000000000015 / 000000030000000000000015
    database size: 75.5MB, database backup size: 31.2MB
    repo1: backup set size: 17.3MB, backup size: 11MB
    backup reference total: 1 full, 1 diff, 1 incr

  incr backup: 20260528-104248F_20260530-020024I
    timestamp start/stop: 2026-05-30 02:00:24+00 / 2026-05-30 02:00:26+00
    wal start/stop: 000000030000000000000019 / 000000030000000000000019
    database size: 75.5MB, database backup size: 48.3KB
    repo1: backup set size: 17.3MB, backup size: 1.1KB
    backup reference total: 1 full, 1 diff, 2 incr

```

Şekil 1. pgBackRest yedek listesi — tam, diferansiyel ve artımlı yedekler boyutlarıyla

6. Felaket Senaryoları ve Zamana Dayalı Kurtarma (PITR)

Yedekleme altyapısının asıl sınavı, gerçek bir veri kaybı karşısında veriyi eksiksiz geri getirebilmesidir. Bu amaçla iki ayrı felaket senaryosu tetiklenmiştir.

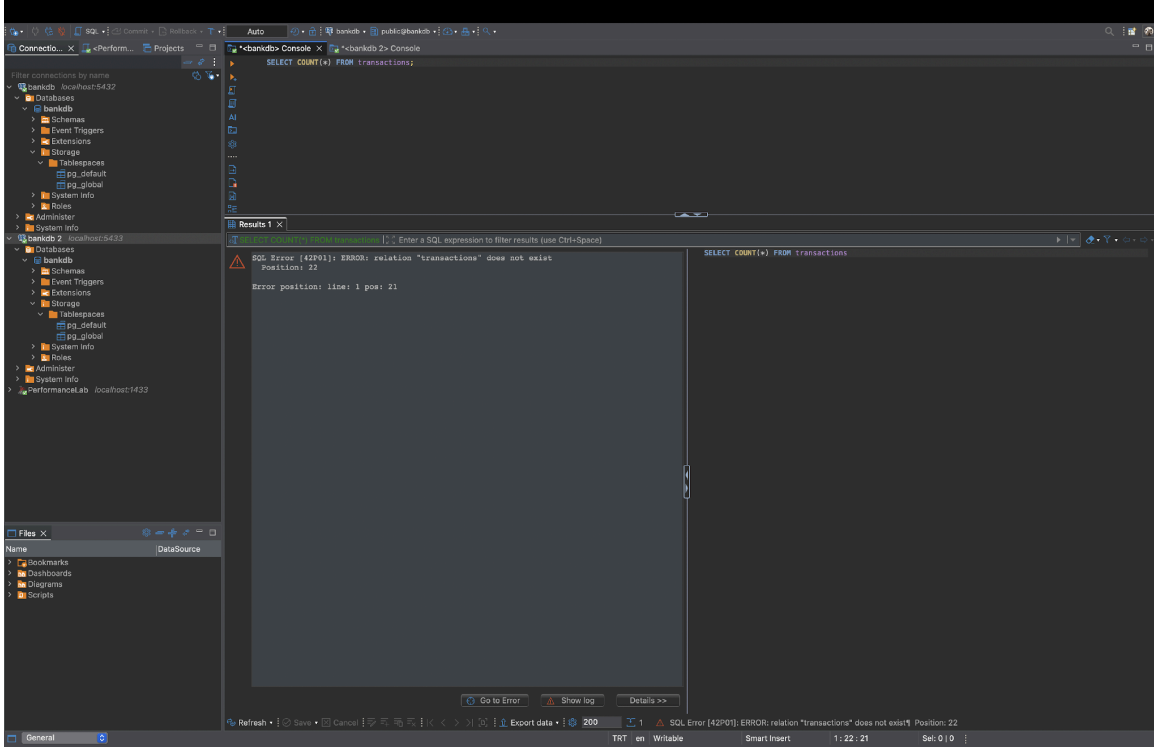
6.1. Felaket A — Yanlışlıkla Toplu Silme

İlk senaryoda, koşul (WHERE) bölümü unutulmuş bir silme sorgusuyla transactions tablosundaki tüm kayıtlar yanlışlıkla silinmiştir. Felaket anından hemen önce oluşturulan bir geri dönüş noktası (restore point) sayesinde, veritabanı tam o ana döndürülerek silinen kayıtların tamamı kurtarılmıştır.

6.2. Felaket B — Kritik Tablonun Düşürülmesi

İkinci senaryoda, yalnızca verisi değil yapısı da kaldırılan bir tablo düşürme (DROP TABLE) işlemi gerçekleştirilmiştir. Bu, silme işleminden daha ağır bir felakettir; çünkü tablonun tanımı

da kaybolur. PITR ile felaket öncesindeki isimli geri dönüş noktasına dönülerek hem tablo yapısı hem de içindeki tüm veriler eksiksiz geri getirilmiştir.



Şekil 2. Felaket anı — kritik tablonun silinmiş/düşürülmüş durumu

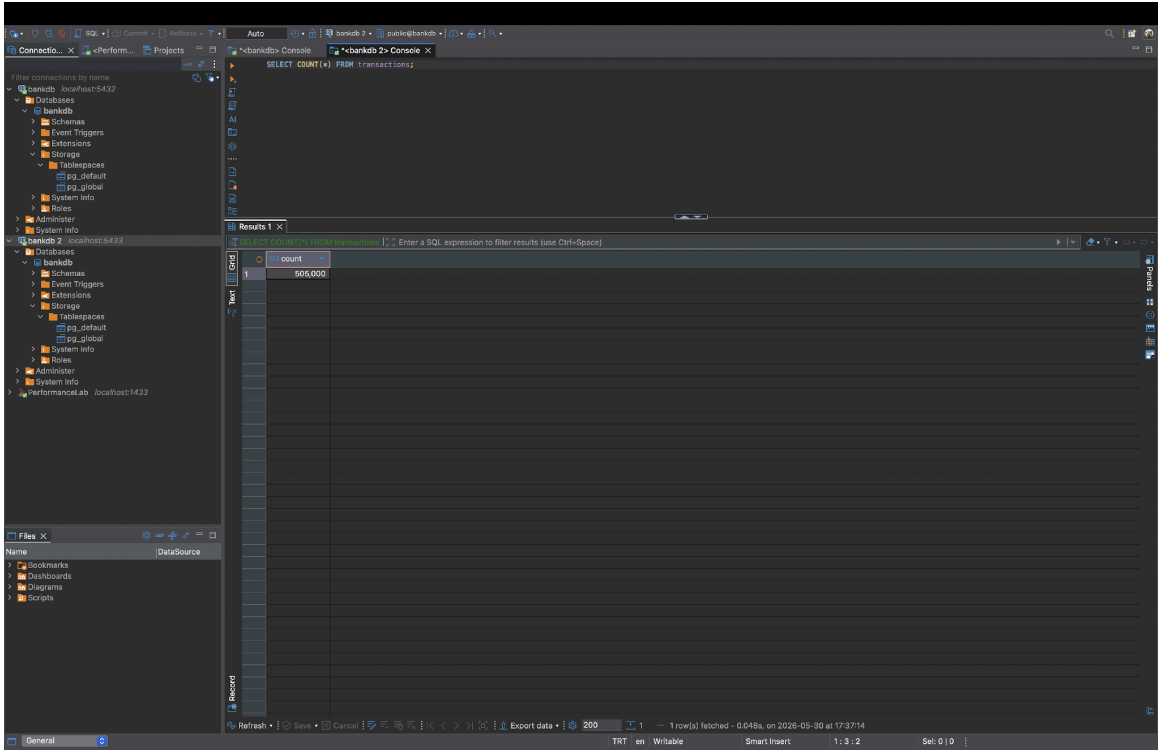
Her iki senaryoda da kurtarma işlemi, felaket anından hemen önceki hedefe (restore to target) dönülerek gerçekleştirilmiş; isimli geri dönüş noktaları sayesinde saat ya da saniye belirsizliğine düşülmeden tam istenen ana geri dönmüştür.

7. Yedeklerin Doğrulanması

Bir yedeğin yalnızca alınmış olması yeterli değildir; gerçekten geri yüklenebilir olduğunun da kanıtlanması gerekir. Bu amaçla, son yedek kullan-at doğrulama konteynerine geri yüklenmiş ve asıl veritabanıyla satır sayıları karşılaştırılmıştır. Bu yaklaşım, asıl (üretim) veritabanına hiç dokunmadan yedeklerin sağlamlığının otomatik olarak denetlenmesini sağlar.

```
2026-05-30 14:42:48.459 P00 INFO: restore command end: completed successfully (245ms)
Starting pg-restore-test container...
[+] start 1/1
✓ Container pg-restore-test Started
Waiting for test postgres to be ready...
Comparing data...
Primary Transaction Count: 505000
Restore Transaction Count: 505000
PASS: Backup restored successfully and data matches!
```

Şekil 3. Otomatik doğrulama — yedekten dönen veri ile asıl veri satır sayılarının eşleşmesi



Şekil 4. Kurtarılan verilerin kontrolü — kayıtların eksiksiz geri geldiği görünümü

8. Elde Edilen Sonuçlar

- Sistem, standart pg_dump yedeklemesinden WAL temelli sürekli arşivlemeye başarıyla geçirilmiştir.
- Kritik veri silme ve tablo düşürme anlarında dahi hiçbir veri kaybı yaşanmadan, veritabanının istenen zamana sorunsuz döndürülebildiği PITR ile kanıtlanmıştır.
- Kullan-at doğrulama konteyneri sayesinde, asıl üretim veritabanı etkilenmeden yedeklerin sağlamlığı otomatik olarak denetlenebilir hâle gelmiştir.

9. Karşılaşılan Problemler ve Çözümleri

9.1. Salt-Okunur Yedek Deposu Hatası

Otomatik doğrulama betiği çalıştırıldığında “salt-okunur dosya sistemi” hatası alınmıştır. Nedeni, yedek deposunun konteynere yalnızca okuma izniyle bağlanmış olmasıydı; oysa pgBackRest doğrulama sırasında bu klasörde geçici dosyalar üretmek ister. Salt-okunur kısıtlaması kaldırılarak sorun giderilmiştir.

9.2. Yanlış Hedefe Geri Yükleme

Geri yükleme sırasında, hedef nokta belirtilmediğinde sistem WAL kayıtlarının en sonuna (yani tablonun düşürüldüğü anın da ilerisine) geri yükleme yapıyor ve “tablo bulunamadı” hatası üretiyordu. Geri yükleme komutuna felaket öncesindeki isimli hedef nokta açıkça verilerek (restore to target) sorun çözülmüştür. Bu, PITR'de hedef belirlemenin ne kadar kritik olduğunu açıkça göstermiştir.

9.3. Kurulum Sırasında Geçici Kesinti

Kurulum sırasında anlık bir ağ kesintisinden ötürü paket yöneticisi hata vermiştir. Konteyner mimarisi durdur-başlat senaryolarına dayanıklı biçimde izole tasarlandığından, işlem yalnızca yeniden çalıştırılarak kaldığı yerden sürdürülmüştür.

10. Sonuç ve Değerlendirme

Bu projede, aktif ve kritik sistemlerde veritabanı operasyonlarının en önemli ayaklarından biri olan yedekleme stratejisinin değeri uygulamalı olarak gösterilmiştir. Yalnızca günlük anlık kopya almanın bir kriz anında yeterli olmadığı; WAL temelli zamana dayalı kurtarma yeteneğine sahip pgBackRest gibi araçların, üretim ortamlarında veri kaybını önlemede temel standart olduğu açıkça ortaya konmuştur. Ayrıca yedeklerin yalnızca alınmasının değil, düzenli olarak geri yüklenerek doğrulanmasının da bir yedekleme stratejisinin ayrılmaz parçası olduğu vurgulanmıştır.

Projenin tüm kodları, çalışabilirlik sınamalarının ardından ayrı bir denetim (audit) sürecinden geçirilmiş; test amacıyla bilinçli olarak zayıf bırakılan ayarların (örneğin yapılandırmadaki düz metin parolalar veya gevşek erişim kuralları) gerçek bir üretim ortamına taşınmadan önce nasıl düzeltilmesi gerektiği, projedeki OPTIMIZATIONS.md belgesinde bir mühendislik farkındalık raporu olarak ayrıca ele alınmıştır.

PROJE 3: VERİTABANI GÜVENLİĞİ VE ERIŞİM KONTROLÜ

1. Projenin Amacı

Bu projenin temel amacı, bir “Hastane Bilgi Sistemi” senaryosu üzerinden, hassas verinin (TC kimlik numarası, tanı, tedavi gibi) korumasız tutulduğu güvensiz bir başlangıç durumundan; katmanlı güvenlik denetimleriyle donatılmış güvenli bir hedef duruma geçişi, somut önce-sonra kanıtlarıyla göstermektir. Sağlık verisi doğası gereği son derece hassas olduğundan, şifreleme ve erişim kısıtlama gibi denetimlerin gerçekçi ve anlamlı biçimde sınanmasına olanak tanır.

Proje kapsamında beş güvenlik eksenini uygulanmıştır: (1) rol tabanlı erişim kontrolü (en az yetki ilkesi), (2) hassas alanların AES-256 ile şifrelenmesi, (3) yetkiye göre veri maskeleye, (4) SQL enjeksiyonuna (SQL Injection) karşı savunma ve (5) tüm hassas erişimlerin denetim (audit) kaydı altına alınması. Her denetim, ilgili kullanıcının neyi yapıp neyi yapamadığı canlı olarak sınanarak doğrulanmıştır.

2. Kullanılan Ortam ve Araçlar

- Oracle Database 23ai Free — Veritabanı yönetim sistemi. Geliştirme makinesinin ARM mimarisinde olması nedeniyle, yalnızca x86 destekleyen ve öykünmede (emulation) sorunlu çalışan eski sürüm yerine, ARM üzerinde doğal çalışan güncel 23ai Free sürümü tercih edilmiştir.
- DBMS_CRYPTO — Oracle'ın yerleşik şifreleme paketi; AES-256 ile kolon bazlı şifreleme için kullanılmıştır.
- Görünüm (VIEW) tabanlı maskeleye — Düşük yetkili kullanıcılara verinin maskeli biçimini sunmak için.
- Unified Auditing — Oracle'ın modern, birleşik denetim altyapısı; hassas tablo erişimlerini kaydeder.
- Python (python-oracledb) — SQL enjeksiyonu gösterimi için; istemci kurulumu gerektirmeyen ince (thin) kipte çalışır.

3. Kullanılan Veri Seti ve Veritabanı

Tüm veri, tekrar üretilebilirlik adına Oracle içinde sentetik olarak üretilmiştir. Şema dört tablodan oluşur ve hassas alanlar bilinçli olarak farklı tablolara dağıtılmıştır.

Çizelge 3. Hastane şeması ve hassas alanlar

Tablo	İçerik	Hassas Alanlar
doctors	Doktorlar (≈ 50 satır)	—
patients	Hastalar (≈ 1.000 satır)	TC kimlik, telefon, adres
medical_records	Tıbbi kayıtlar (≈ 5.000 satır)	tanı, tedavi
appointments	Randevular (≈ 5.000 satır)	—

Bu projede kritik olan veri hacmi değil, hassas alanların farklı yetki düzeylerinde nasıl korunduğudur.

4. Başlangıç Durumu — Güvensiz Sistem

Sistem, kasıtlı olarak güvensiz bir durumla kurulmuştur; böylece her zafiyetin sonradan nasıl kapatıldığı önce-sonra karşılaştırmasıyla gösterilebilir. Başlangıçtaki açıklar şunlardır:

- TC kimlik numarası düz metin olarak saklanıyor ve şemaya erişen herkes tarafından okunabiliyordu.
- Tanı ve tedavi gibi hassas tıbbi veriler hiçbir kısıtlama olmadan açıldı.
- Rol ayrımı yoktu; şifreleme yoktu; hiçbir denetim politikası etkin değildi.
- Uygulama tarafındaki hasta arama sorgusu, kullanıcı girdisini doğrudan SQL metnine gömerek enjeksiyona açık biçimde yazılmıştı.

```
bora@bora-MacBook-Air project-3-security % ./scripts/run_sql.sh 'hospital_app/"Hospital#2026app"@localhost:1521/FREEDB1' sql/01_init_schema.sql
./scripts/run_sql.sh 'hospital_app/"Hospital#2026app"@localhost:1521/FREEDB1' sql/02_insecure_baseline.sql
PL/SQL procedure successfully completed.

Table created.

Table created.

Table created.

Table created.

50 rows created.

1000 rows created.

5000 rows created.

5000 rows created.

Commit complete.

>>> Sema + veri yüklendi:

TABLE          ADET
-----
doctors         50
patients       1000
medical_records 5000
appointments    5000

=== [GUVENSIZ] TC kimlik DUZ METIN olarak okunabiliyor ===

PATIENT_ID FULL_NAME                                NATIONAL_ID PHONE
-----
318 Vhfelc Ivamevww                                79981940759 05998504654
319 Majyfm Ptfbsgxj                                19573153509 05925403741
320 Tbzviu Xlfmelm                                  53364090688 05153394396
321 Zkmupt Npztlant                                97188584296 05632659099
322 Veiyuf Somifhce                                46744129795 05772114753

=== [GUVENSIZ] Hassas tıbbi veri (tani/tedavi) acik ===

RECORD_ID FULL_NAME
-----
DIAGNOSIS
-----
TREATMENT
-----
691 Rjsdst Mmtshyjn
Bronsit
Antibiyotik
692 Apamoc Twkbqujf
Anemi
Demir takviyesi
693 Zihmtr Lxjnsuw
Gastrit
Antasit
694 Xddsgt Weguhjsr
Astim
Inhaler
695 Ukvqfv Gxpgbdgo
Grip
Istirahat

=== [GUVENSIZ] national_id duz metin bir kolon (sifreli degil) ===

COLUMN_NAME
-----
DATA_TYPE          DATA_LENGTH
-----
NATIONAL_ID
VARCHAR2            11

>>> Durum: rol yok, sifreleme yok, audit kapali. (Bolum 1-4'te kapatilacak)
```

Şekil 5. Güvensiz başlangıç — TC kimlik numarasının düz metin olarak okunabilmesi

Aşağıdaki görüntüde, şifreli kolonun ham (okunamaz) hâli ile yetkili kullanıcının çözülmüş görünümü yan yana sunulmaktadır.

```
|bora@bora-MacBook-Air project-3-security % ./scripts/run_sql.sh 'hospital_app"/Hospital#2026app"@localhost:1521/FREEPDB1' sql/04_encryption_dbms_crypto.sql
PL/SQL procedure successfully completed.

Function created.

Function created.

1000 rows updated.

Commit complete.

PL/SQL procedure successfully completed.

Grant succeeded.

=== Sifreli kolon ham hali (anlamsız RAW) ===
PATIENT_ID
-----
NATIONAL_ID_ENC
-----
318
E7DEB3DDC339AF57DA32A40B8200FC6A
319
5D59C5FA541995AEEC43A27E58664FD6
320
B0718A144F940B7CA67831745F26701C

=== Yetkili cozum (fn_decrypt_nid) ===
PATIENT_ID
-----
TC
-----
318
79981940759
319
19573153509
320
53364090688

>>> national_id artik AES-256 ile sifreli.
```

Şekil 7. Şifreleme — diskteki şifreli (RAW) veri ile yetkili kullanıcının çözülmüş görünümü

Not olarak, ders dokümanında geçen TDE (saydam veri şifreleme) yalnızca Oracle'ın ücretli kurumsal sürümünde bulunduğundan; aynı amaca (hassas alanın diskte korunması) hizmet eden ve ücretsiz sürümde çalışan kolon bazlı DBMS_CRYPTO şifrelemesi tercih edilmiştir.

5.3. Yetkiye Göre Veri Maskeleye

Şifreleme veriyi diskte korurken, maskeleye aynı veriyi kullanıcının yetkisine göre farklı gösterir. Resepsiyonist gibi düşük yetkili kullanıcıların ana tabloya erişimi tamamen kapatılmış; bunun yerine TC kimliği XXX-XX-1234 biçiminde, telefonu kısmen gizleyen bir maskeli görünüm (view) sunulmuştur. Böylece resepsiyonist işini yapabilir ancak hassas veriyi tam olarak göremez.

```

bora@bora-MacBook-Air project-3-security % ./scripts/run_sql.sh 'hospital_app/"Hospital#2026app"@localhost:1521/FREEDB1' sql/05_masking_views.sql
./scripts/run_sql.sh 'system/"Admin#2026pass"@localhost:1521/FREEDB1' sql/06_unified_audit.sql
./scripts/run_sql.sh 'reception1/"Recepti#2026"@localhost:1521/FREEDB1' sql/07_test_reception.sql

View created.

Grant succeeded.

=== Maskeli gorunum (resepsiyonistin gorecegi) ===

PATIENT_ID FULL_NAME
-----
PHONE_MASKED BLO
-----
318 Vhfelc Ivemevvw XXX-XX-0759
0599****54 AB+
319 Majyfm Ptfbsgxj XXX-XX-3509
0592****41 AB-
320 Tbzviu Xlfmdelm XXX-XX-0688
0515****96 A+

PATIENT_ID FULL_NAME
-----
PHONE_MASKED BLO
-----
321 Zkmupt Npztlant XXX-XX-4296
0563****99 A-
322 Veiuf Somifhce XXX-XX-9795
0577****53 B+

>>> Maskeleme view'i hazir; resepsiyonist ana tabloyu goremez.

PL/SQL procedure successfully completed.

PL/SQL procedure successfully completed.

Audit policy created.

Audit succeeded.

=== Aktif audit politikolari ===

POLICY_NAME
-----
ENABLED_OPTION
-----
POL_PATIENT_ACCESS
BY USER

>>> Unified Audit politikasi aktif.
=== [BEKLENEN: RED] reception1 -> medical_records ===
SELECT * FROM hospital_app.medical_records WHERE ROWNUM <= 1
*
ERROR at line 1:
ORA-00942: table or view "HOSPITAL_APP"."MEDICAL_RECORDS" does not exist
Help: https://docs.oracle.com/error-help/db/ora-00942/

=== [BEKLENEN: RED] reception1 -> patients (ana tablo) ===
SELECT * FROM hospital_app.patients WHERE ROWNUM <= 1
*
ERROR at line 1:
ORA-00942: table or view "HOSPITAL_APP"."PATIENTS" does not exist
Help: https://docs.oracle.com/error-help/db/ora-00942/

=== [BEKLENEN: MASKELI] reception1 -> v_patients_masked ===

PATIENT_ID FULL_NAME
-----
PHONE_MASKED BLO
-----
318 Vhfelc Ivemevvw XXX-XX-0759
0599****54 AB+
319 Majyfm Ptfbsgxj XXX-XX-3509
0592****41 AB-
320 Tbzviu Xlfmdelm XXX-XX-0688
0515****96 A+
321 Zkmupt Npztlant XXX-XX-4296
0563****99 A-
322 Veiuf Somifhce XXX-XX-9795
0577****53 B+

```

Şekil 8. Maskeleme — resepsiyonistin gördüğü maskeli TC ve telefon

Oracle, kolon bazlı SELECT yetkisini desteklemediğinden (kolon bazlı yetki yalnızca yazma işlemleri için geçerlidir), belirli kolonları gizlemenin yolu olarak görünüm (view) kullanılmış ve bu yaklaşım maskeleme ile birleştirilmiştir.

5.4. SQL Enjeksiyonuna Karşı Savunma

Aynı kötü niyetli kullanıcı girdisi iki ayrı yöntemle çalıştırılarak fark kanıtlanmıştır. Girdinin doğrudan SQL metnine gömüldüğü zafiyetli sürümde, klasik bir enjeksiyon ifadesi tüm 1.000 hasta kaydının sızdırılmasına yol açmıştır. Aynı girdi, parametrelili sorgu (bind variable)

kullanan güvenli sürümde ise hiçbir kayıt döndürmemiş; çünkü girdi artık çalıştırılabilir kod olarak değil, salt veri olarak işlenmiştir.

```
bora@bora-MacBook-Air project-3-security % python3 -m venv .venv && source .venv/bin/activate
pip install -r app/requirements.txt
python app/injection_demo.py
Requirement already satisfied: oracledb==2.0 in ./venv/lib/python3.14/site-packages (from -r app/requirements.txt (line 1)) (4.0.1)
Requirement already satisfied: cryptography==3.2.1 in ./venv/lib/python3.14/site-packages (from oracledb==2.0->-r app/requirements.txt (line 1)) (48.0.0)
Requirement already satisfied: typing_extensions==4.14.0 in ./venv/lib/python3.14/site-packages (from oracledb==2.0->-r app/requirements.txt (line 1)) (4.15.0)
Requirement already satisfied: cffi==2.0.0 in ./venv/lib/python3.14/site-packages (from cryptography==3.2.1->oracledb==2.0->-r app/requirements.txt (line 1)) (2.0.0)
Requirement already satisfied: pycparser in ./venv/lib/python3.14/site-packages (from cffi==2.0.0->cryptography==3.2.1->oracledb==2.0->-r app/requirements.txt (line 1)) (3.0)

Veritabanında toplam 1000 hasta var.
Saldırgan girdisi (payload): "x' OR '1'='1"

=====
1) ZAFİYETLİ SORGU
=====
[zafiyetli SQL] SELECT patient_id, full_name FROM hospital_app.patients WHERE full_name = 'x' OR '1'='1'
-> Donen kayıt sayısı: 1000
-> !! TUM hastalar sizdi (injection BASARILI)

=====
2) GUVENLİ SORGU (bind variable)
=====
[güvenli SQL] SELECT patient_id, full_name FROM hospital_app.patients WHERE full_name = :name | bind name = "x' OR '1'='1"
-> Donen kayıt sayısı: 0
-> Injection ETKİSİZ (girdi veri olarak işlendi)
```

Şekil 9. SQL enjeksiyonu — zafiyetli sorgu tüm kayıtları sızdırır, güvenli sorgu engeller

5.5. Denetim (Audit) Kayıtları ve Görev Ayrılığı

Hassas tablolara yapılan tüm erişimler, Unified Auditing politikasıyla kayıt altına alınmıştır. Böylece kimin, ne zaman, neye, hangi sonuçla eriştiği geriye dönük izlenebilir hâle gelmiştir. Denetçi kullanıcısı, görev ayrılığı (segregation of duties) ilkesi gereği hasta verisini göremez; yalnızca denetim kayıtlarını okuyabilir. Bu, denetimi yapanın denetlediği veriye erişememesi gerektiği yönündeki kurumsal güvenlik ilkesinin uygulamasıdır.

```
bora@bora-MacBook-Air project-3-security % ./scripts/run_sql.sh 'auditor1/"Auditor1#2026"@localhost:1521/FREEPDB1' sql/09_test_auditor.sql
=== [BEKLENEN: RED] auditor1 -> hasta verisi ===
SELECT * FROM hospital_app.patients WHERE ROWNUM <= 1
*
ERROR at line 1:
ORA-00942: table or view "HOSPITAL_APP"."PATIENTS" does not exist
Help: https://docs.oracle.com/error-help/db/ora-00942/

=== [BEKLENEN: BASARILI] auditor1 -> audit trail erişimi ===

AUDIT_KAYIT_SAYISI
=====
12
```

Şekil 10. Görev ayrılığı — denetçi hasta verisine erişemez, yalnızca denetim izini okur


```

|bora@bora-MacBook-Air project-3-security % ./scripts/run_sql.sh 'system/"Admin#2026pass"@localhost:1521/FREEDB1' sql/10_audit_review.sql
=== Audit trail: kim / ne zaman / neye / sonuc ===
(return_code: 0 = basarili erisim, <>0 = engellendi/hata)

ZAMAN      DBUSERNAME
-----
ACTION_NAME
-----
OBJECT_NAME
-----
RETURN_CODE
-----
19:30:54 AUDITOR1
SELECT
PATIENTS
2004
19:30:44 DR_HOUSE
SELECT
PATIENTS
0
19:30:07 RECEPTION1
SELECT
PATIENTS
0
19:30:07 RECEPTION1
SELECT
PATIENTS
2004
19:30:07 RECEPTION1
SELECT
MEDICAL_RECORDS
2004
16:32:44 AUDITOR1
SELECT
PATIENTS
2004
16:32:44 DR_HOUSE
SELECT
PATIENTS
0
16:32:36 AUDITOR1
SELECT
PATIENTS
2004
16:32:36 DR_HOUSE
SELECT
PATIENTS
0
16:32:36 RECEPTION1
SELECT
PATIENTS
0
16:32:36 RECEPTION1
SELECT
PATIENTS
2004
ZAMAN      DBUSERNAME
-----
ACTION_NAME
-----
OBJECT_NAME
-----
RETURN_CODE
-----
16:32:36 RECEPTION1
SELECT
MEDICAL_RECORDS
2004

12 rows selected.

>>> Audit incelemesi tamam. (rc=0 dr_house basarili; rc<>0 reception1/auditor1 engellendi)

```

Şekil 11. Denetim izi — kim, ne zaman, neye, hangi sonuçla erişti

6. Elde Edilen Sonuçlar

Tüm betikler, sıfırdan kurulan temiz bir ortam üzerinde hatasız çalıştırılmış ve her güvenlik denetiminin etkisi doğrulanmıştır. Önce-sonra karşılaştırması aşağıdaki çizelgede özetlenmiştir.

Çizelge 4. Güvenlik denetimlerinin önce/sonra karşılaştırması

Denetim	Güvensiz Başlangıç	Güvenli Sonuç
Şifreleme	TC düz metin	Diskte AES-256 şifreli; düz metin kolon silindi
Maskelme	Herkes tam TC'yi görür	Resepsiyonist yalnızca maskeli TC görür
Erişim kontrolü	Tek, aşırı yetkili kullanıcı	Yetkisiz kullanıcılar ana tablolara erişemez
Görev ayrılığı	—	Denetçi veriyi göremez, yalnızca kayıtları okur
SQL enjeksiyonu	Tüm kayıtlar sızar	Parametrelili sorguyla sıfır kayıt döner

Denetim kayıtlarının incelenmesinde, doktorun hasta tablosuna erişiminin başarılı (dönüş kodu sıfır), resepsiyonist ve denetçinin ana tablolara erişim denemelerinin ise reddedildiği (sıfırdan farklı dönüş kodu) açıkça görülmüştür. Bu, hem erişim kontrolünün hem de denetim mekanizmasının birlikte doğru çalıştığını kanıtlar.

7. Karşılaşılan Problemler ve Çözümleri

7.1. Şifreleme Yetkisinin Verilmesi

Şifreleme paketinin kullanım yetkisi, sıradan yönetici kullanıcısıyla verilmeye çalışıldığında hata alınmıştır. Oracle 23ai'de bu tür sistem paketlerinin yetkisi yalnızca en üst düzey yönetici (SYS) tarafından verilebildiğinden, ilgili yetkilendirme adımı bu kullanıcıyla yürütülmüştür.

7.2. Test Bağlantılarının Güvenilirliği

Tek bir oturum içinde art arda kullanıcı değiştirildiğinde, varsayılan rollerin tutarsız biçimde etkinleştiği görülmüştür. Bu nedenle her güvenlik testi, ilgili kullanıcı için ayrı bir bağlantıyla çalıştırılarak sonuçların güvenilirliği sağlanmıştır.

7.3. Sürüm Kaynaklı Uyarlamalar

Eski Oracle sürümlerinde denetim kayıtlarını sorgulamadan önce gereken elle boşaltma (flush) adımının, 23ai sürümünde hem gereksiz olduğu (kayıtlar anında görünür) hem de sıradan yöneticiye kapalı olduğu görülmüş ve bu adım kaldırılmıştır.

8. Sonuç ve Değerlendirme

Bu projede, veritabanı güvenliğinin tek bir önlemle değil, katmanlı bir savunma (defense in depth) yaklaşımıyla sağlandığı kanıtlanmıştır. Şifreleme veriyi diskte ve yedekte korurken, maskeleyme aynı veriyi yetki düzeyine göre farklı gösterir; rol ayrımı kullanıcının erişim alanını daraltır; parametrelili sorgular uygulama katmanındaki en yaygın saldırı olan SQL enjeksiyonunu etkisiz kılar; denetim ise tüm bu erişimleri geriye dönük izlenebilir hâle getirir. Özellikle görev ayrılığı ilkesinin (denetçinin hasta verisini görmeden yalnızca kayıtları okuyabilmesi) gerçek dünyadaki kurumsal güvenlik politikalarının çekirdeğini oluşturduğu açıkça gösterilmiştir.

PROJE 4: VERİTABANI YÜK DENGELEME VE DAĞITIK YAPILAR

1. Projenin Amacı

Bu projenin temel amacı, tek bir veritabanı sunucusuna bağımlılığın (Single Point of Failure, SPOF) yarattığı hizmet kesintisi riskini ortadan kaldıran; coğrafi olarak dağıtık ve yüksek erişilebilir (High Availability, HA) bir PostgreSQL kümesi kurmaktır. Üç farklı sağlayıcıda yer alan üç gerçek sunucu, şifreli bir özel ağ üzerinden birbirine bağlanmış; bu altyapı üzerinde verinin gerçek zamanlı çoğaltılması, sunucu arızasında otomatik lider seçimi ve okuma/yazma yükünün uygun düğümlere dağıtılması somut kanıtlarla gösterilmiştir.

Bu çalışma, dersin başlığı olan “Ağ Tabanlı Paralel Dağıtım Sistemleri” kavramına en doğrudan karşılık gelen projedir: çoğaltma (replication) ve devralma (failover) işlemleri tek bir makine üzerinde benzetim yoluyla değil, birbirinden bağımsız gerçek makineler arasında ve gerçek ağ üzerinden gerçekleştirilmektedir. Bu nedenle proje yalnızca veritabanı yönetimini değil, ağ güvenliği, dağıtık konsensüs ve sistem yönetimi konularını da bir arada içermektedir.

1.1. Giderilmesi Hedeflenen Problemler

- Tek sunucu bağımlılığı: tek düğümün çökmesi durumunda hizmetin tamamen kesilmesi.
- Otomatik devralma eksikliği: arıza anında insan müdahalesi olmadan yeni liderin seçilememesi.
- Okuma yükünün tek düğüm üzerinde toplanması ve kaynakların verimsiz kullanılması.
- Yük dengeleyicinin kendisinin tek arıza noktası hâline gelmesi riski.

2. Kullanılan Ortam ve Araçlar

Sistem, her biri belirli bir katmandan sorumlu olan beş ana bileşenin uyumlu çalışmasıyla kurulmuştur. Bileşenler ve görevleri aşağıda açıklanmıştır.

- PostgreSQL 16.14 — Veri katmanı. Düğümlerden biri yazma yetkili birincil (primary), diğeri salt-okunur yedek (replica) olarak çalışır.
- Patroni 4.1.3 — Yüksek erişilebilirlik orkestratörü. PostgreSQL süreçlerini, küme durumuna göre yönetir; lider seçimini, yedek bağlamayı ve arıza sonrası devralmayı otomatikleştirir.
- etcd 3.5.21 — Dağıtık karar (konsensüs) deposu. “Şu an birincil hangi düğüm?” bilgisini çoğunluk oyuyla tutar; Patroni kararlarını bu depo üzerinden alır.
- HAProxy 2.9 — Yük dengeleyici. Uygulamadan gelen yazma/okuma isteklerini, Patroni'nin sağlık arabirimini sorgulayarak doğru düğüme yönlendirir.
- WireGuard — Şifreli özel ağ (mesh). Üç sunucu arasındaki tüm küme trafiğini, internete hiç açılmadan şifreli bir tünelden taşır.

Üç sunucu ve üstlendikleri roller aşağıdaki çizelgede özetlenmiştir. Sunucuların kasıtlı olarak üç farklı sağlayıcıdan seçilmesi, gerçek bir dağıtık sistemin coğrafi ve yönetsel bağımsızlık ilkesini yansıtmak amacını taşır.

Çizelge 5. Küme düğümleri ve rolleri

Düğüm	Ağ Adresi	Donanım / Sağlayıcı	Üstlenilen Rol
Contabo	10.10.0.1	x86_64, 8 GB, Debian 13	PostgreSQL + Patroni + etcd + HAProxy
AWS t4g.small	10.10.0.2	ARM64, 2 GB, Debian 13	PostgreSQL + Patroni + etcd + HAProxy

Tanık Düşüm	10.10.0.3	x86_64, mevcut sunucu	etcd tanık (yalnızca çoğunluk için)
-------------	-----------	-----------------------	-------------------------------------

2.1. Çok Mimari (Heterojen) Donanım

Düşümlerin işlemci mimarileri farklıdır: Contabo x86_64, AWS ise ARM64 (Graviton) tabanlıdır. Kullanılan tüm konteyner görüntüleri çok mimari (multi-architecture) olduğundan, her düşüm kendi mimarisinde doğal (native) biçimde çalışmış; bu durum çoğaltma veya devralma sürecinde herhangi bir uyumsuzluğa yol açmamıştır. PostgreSQL'in mimariden bağımsız ikili (binary) çoğaltma protokolü, farklı işlemci mimarilerine sahip düşümler arasında sorunsuz çalışmaktadır.

3. Sistem Mimaris ve Ağ Altyapısı

Mimari üç katmandan oluşur: en altta düşümleri birbirine bağlayan şifreli özel ağ, onun üzerinde kararları veren konsensüs katmanı (etcd) ve en üstte bu kararlara göre veriyi yöneten uygulama katmanı (PostgreSQL + Patroni + HAProxy). Bu bölümde her katman sırasıyla açıklanmıştır.

3.1. WireGuard Şifreli Özel Ağ

Üç sunucu arasındaki tüm küme trafiği (çoğaltma, etcd ve Patroni haberleşmesi), 10.10.0.0/24 adres bloğunda kurulan bir WireGuard özel ağı üzerinden taşınmaktadır. Her düşüm diğer ikisini doğrudan tanır; yani ağ, yıldız değil tam örgü (full mesh) topolojisindedir. Bu sayede hiçbir veritabanı veya yönetim portu doğrudan internete açılmamış, bütün hassas trafik şifreli tünel içinde tutulmuştur. Kurulum, sunuculardan birinde hâlihazırda çalışan başka bir VPN hizmetine (OpenVPN) dokunulmadan, ayrı bir ağ arabirimi (wg0) üzerinden yapılmıştır.

AWS düşümü, ağ adresi çevirisi (NAT) arkasında bulunduğundan, tünelin sürekli açık kalması için yapılandırmaya düzenli canlılık sinyali (PersistentKeepalive = 25 saniye) eklenmiştir. Aşağıdaki çıktı, üç düşüm arasındaki karşılıklı el sıkışmaların (handshake) gerçekleştiğini ve tünelin etkin olduğunu göstermektedir.

```
[bora@bora-MacBook-Air project-4-load-balancing % ssh contabo 'sudo wg show'
interface: wg0
  public key: ta6svqbd9q1KU8cE7o3PiTvf5fC7LLb0SGckcIJXByE=
  private key: (hidden)
  listening port: 51820

peer: 0VUrvRsJPACbtu7M47Yf/XW1wSARmaItkWAHzB2/aG8=
  endpoint: 193.35.154.38:51820
  allowed ips: 10.10.0.3/32
  latest handshake: 1 minute, 40 seconds ago
  transfer: 161.14 MiB received, 160.55 MiB sent
  persistent keepalive: every 25 seconds

peer: F57MtAPry5Qy9K1KGs84CHPR6NtLhnLsmMpEdmnJBmM=
  endpoint: 13.49.125.118:51820
  allowed ips: 10.10.0.2/32
  latest handshake: 1 minute, 52 seconds ago
  transfer: 172.58 MiB received, 194.98 MiB sent
  persistent keepalive: every 25 seconds
```

Şekil 12. WireGuard özel ağı — üç düşüm arasında karşılıklı el sıkışma (handshake)

3.2. etcd Üç Düşümlü Konsensüs Kümesi

Dağıtık karar katmanı, üç düşümde çalışan etcd örneklerinden oluşur. Her örnek yalnızca özel ağ adresleri üzerinden haberleşecek biçimde yapılandırılmış; istemci (2379) ve eş (2380) portları internete kapalı tutulmuştur. etcd, RAFT konsensüs algoritmasını kullanarak küme durumunu tüm düşümlerde tutarlı biçimde saklar. Aşağıdaki çıktı, üç düşümün de sağlıklı olduğunu ve kümenin çoğunluk koşulunu sağladığını göstermektedir.

```
bora@bora-MacBook-Air project-4-load-balancing % ssh contabo 'sudo docker exec etcd etcdctl --endpoints=http://10.10.0.1:2379,http://10.10.0.2:2379,http://10.10.0.3:2379 endpoint health --cluster'
```

Şekil 13. etcd küme sağlığı — üç düşümün tamamı “healthy”, RAFT dizinleri eşitlenmiş

3.3. Neden Üç Düşüm? — Çoğunluk (Quorum) Gereksinimi

Patroni, hangi düşümün birincil olduğuna etcd üzerindeki çoğunluk oyu ile karar verir. İki düşümlü bir kurulumda düşümlerden biri çöktüğünde geriye kalan tek düşüm çoğunluğu sağlayamaz (bir, ikinin yarısından fazla değildir) ve otomatik devralma kilitlenir. Bu, iki düşümlü HA kurulumlarının klasik tuzağıdır. Sorunu çözmek için, üzerinde PostgreSQL çalışmayan, yalnızca etcd barındıran hafif bir “tanık (witness)” düşüm eklenmiştir. Bu düşüm yaklaşık 150 MB bellek tüketir ve veri taşımaz; tek görevi oylamada üçüncü oyu sağlamaktır. Böylece bir veritabanı düşümü çöксе dahi üç düşümden ikisi (2/3) ayakta kalarak çoğunluğu korur ve devralma sağlıklı biçimde işler.

3.4. Kurulum Adımları

Kurulum, katmanların birbirine bağımlılığı gözetilerek belirli bir sırada yapılmıştır. Aşağıda adımlar, ortama özgü ayrıntılara (adresler, parolalar) girilmeden mantıksal akışıyla özetlenmiştir; tüm yapılandırma dosyaları, gizli bilgiler ayıklanmış örnek şablonlar (.example) hâlinde depoda bulunmaktadır.

- Şifreli ağ: Her sunucuda WireGuard anahtar çifti üretilir, özel anahtar sunucuda kalır; düşümler birbirini ortak (public) anahtarlarla tanıyarak tam örgü ağı kurar.
- Konsensüs: etcd önce tek düşümle başlatılır, ardından diğer düşümler öğrenen üye olarak eklenip tam üyeliğe terfi ettirilerek üç düşümlü küme oluşturulur.
- Veritabanı: İki veritabanı düşümünde Patroni + PostgreSQL 16 konteyneri çalıştırılır; ilk başlayan düşüm birincil olur, ikincisi otomatik olarak yedek hâline gelir.
- Yük dengeleme: Her iki veritabanı düşümünde HAProxy çalıştırılır ve Patroni'nin sağlık arabirimini izleyerek yazma/okuma yönlendirmesini yapacak biçimde yapılandırılır.
- Doğrulama: Hazır betiklerle (örneğin küme durumu ve devralma sınaması) tüm katmanların birlikte doğru çalıştığı denetlenir.

4. Akıslı Çoğaltma (Streaming Replication)

Patroni, kümeyi ayağa kaldırırken düşümlerden birini birincil (Leader, yazma yetkili), diğerini ise yedek (Replica, salt-okunur) olarak yapılandırır. Birincil düşümde gerçekleşen her değişiklik, PostgreSQL'in ön-yazım günlüğü (Write-Ahead Log, WAL) akışı yoluyla yedek düşüme neredeyse anında aktarılır. Patroni yapılandırmasında bellek ayarları, kümedeki en küçük düşüm olan AWS (yaklaşık 1,5 GB kullanılabilir bellek) profiline göre belirlenmiştir; çünkü devralma sonrasında her iki düşüm de birincil rolünü üstlenebilmelidir. Bu nedenle paylaşılan tampon belleği (shared_buffers) gibi parametreler her iki düşümde özdeş ve ölçülü tutulmuştur.

```
bora@bora-MacBook-Air project-4-load-balancing % ssh contabo 'sudo docker exec patroni patronictl -c /etc/patroni/patroni.yml list'
```

Member	Host	Role	State	TL	Receive LSN	Lag	Replay LSN	Lag
aws	10.10.0.2	Leader	running	2				
contabo	10.10.0.1	Replica	streaming	2	0/3087D20	0	0/3087D20	0

Şekil 14. patronictl list çıktısı — Leader ve Replica düğümleri, çoğaltma gecikmesi (Lag) sıfır

Çoğaltmanın doğruluğu somut olarak sınanmıştır. Birincil düğümde bir tabloya yeni kayıt eklendiğinde, aynı kaydın yedek düğümde anında görüldüğü; buna karşın yedek düğüme doğrudan yazma denemesinin “salt-okunur işlem” hatasıyla reddedildiği gözlemlenmiştir. Bu çift yönlü kanıt, hem verinin başarıyla çoğaltıldığını hem de yedek düğümün veri bütünlüğünü koruyacak biçimde yalnızca okumaya açık olduğunu gösterir.

```
bora@bora-MacBook-Air project-4-load-balancing % echo "=== PRIMARY (AWS) yazma ===" && ssh -i ~/Documents/borak-aws-key.pem admin@ec2-13-49-125-118.eu-north-1.compute.amazonaws.com "sudo docker exec -e PGPASSWORD=$PGW patroni psql -h 10.10.0.2 -U postgres -c '\INSERT INTO accounts(account_balance, Valide) VALUES(1, RETURNING id,owner);'; echo "=== REPLICA (Contabo) okuma + red ===" && ssh contabo 'sudo docker exec -e PGPASSWORD=$PGW patroni psql -h 10.10.0.1 -U postgres -c 'SELECT count(*) FROM accounts; -- \INSERT INTO accounts(account_balance, Valide) VALUES(1, 0);'"; echo "=== PRIMARY (AWS) yazma ==="
```

```
id | owner
---|-----
10 | owner
(1 row)

INSERT 0 1
=== REPLICA (Contabo) okuma + red ===
count
-----
      0
(1 row)

ERROR: cannot execute INSERT in a read-only transaction
```

Şekil 15. Çoğaltma kanıtı — birincilde yazılan veri yedekte görünür, yedek yazmaya kapalıdır

5. Otomatik Devralma (Failover)

Projenin en kritik yeteneği, birincil sunucu çöktüğünde kümenin kendini insan müdahalesi olmadan toparlamasıdır. Patroni bu mekanizmayı, etcd üzerinde tuttuğu sürekli bir “liderlik kilidi” (leader lock) ile gerçekleştirir: birincil düğüm, belirli aralıklarla (yapılandırmada 30 saniyelik yaşam süresi, TTL) bu kilidi yeniler. Birincil çökerse kilit yenilenemez ve süresi dolar; bunu fark eden yedek düğümler, etcd çoğunluğu üzerinden aralarında yeni lideri seçer.

5.1. Devralma Senaryosu

Sinama, çalışan birincil düğümün kasıtlı olarak durdurulmasıyla yapılmıştır. Liderlik kilidinin süresi dolduğunda, ayakta kalan veritabanı düğümü ile tanık düğümün oluşturduğu 2/3 çoğunluk kararıyla kalan düğüm otomatik olarak yeni birincil seçilmiştir. Bu işlem sırasında hiçbir yönetici komutu girilmemiştir. Zaman çizgisi (timeline) değerinin bir artması, PostgreSQL düzeyinde yeni bir tarihsel kolun başladığını, yani devralmanın gerçekleştiğini doğrular.

```
bora@bora-MacBook-Air project-4-load-balancing % echo "==== ONCE #####" && ssh contabo 'sudo docker exec patroni patronictl -c /etc/patroni/patroni.yml list' && ssh -i ~/Documents/borak-aws-key.pem admin@ec2-13-49-125-118.eu-north-1.compute.amazonaws.com 'sudo docker st op patroni' && echo "primary durduruldu, 5sn ---" && sleep 5s && echo "#### SONRA ####" && ssh contabo 'sudo docker exec patroni patronictl -c /etc/patroni/patroni.yml list' && ssh -i ~/Documents/borak-aws-key.pem admin@ec2-13-49-125-118.eu-north-1.compute.amazonaws.com 'sudo docker start patroni'
```

```
==== ONCE #####
Cluster: pg-cluster (7645850983892447264)
Member | Host | Role | State | TL | Receive LSN | Lag | Replay LSN | Lag
---|---|---|---|---|---|---|---|---
aws | 10.10.0.2 | Leader | running | 2 | | | | 
contabo | 10.10.0.1 | Replica | streaming | 2 | 0/308D0C8 | 0 | 0/308D0C8 | 0

patroni
=== primary durduruldu, 5sn ---
==== SONRA ====
Cluster: pg-cluster (7645850983892447264)
Member | Host | Role | State | TL | Receive LSN | Lag | Replay LSN | Lag
---|---|---|---|---|---|---|---|---
aws | 10.10.0.2 | Leader | running | 2 | | | | 
contabo | 10.10.0.1 | Leader | running | 2 | | | | 
patroni
```

Şekil 16. Otomatik devralma — birincil durdurulunca liderlik el değiştirir, zaman çizgisi artar

5.2. Geri Katılım (Failback)

Devralmanın ardından durdurulan eski birincil düğüm yeniden başlatıldığında, küme onu otomatik olarak yeni birincilin yedeği durumunda tekrar kabul etmiştir. Eski birincilin, kendisi çevrimdışıyken yeni birincilde oluşan değişikliklerle hızlıca uyumlanması için Patroni'nin pg_rewind yeteneği etkinleştirilmiştir. Devralma sırasında yeni birincile yazılan verilerin de geri dönen düğüme eksiksiz çoğaltıldığı doğrulanmış; böylece devralma ve geri katılım döngüsü patroni kaybı olmadan tamamlanmıştır.

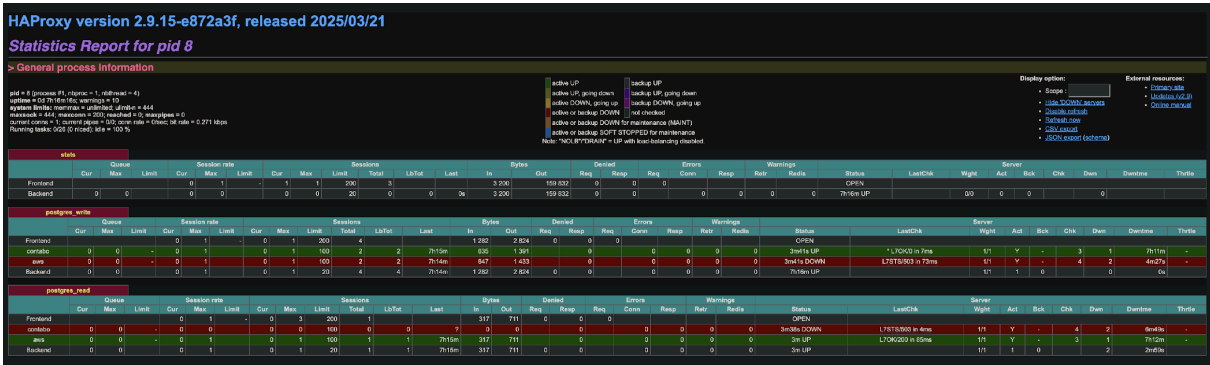
6. Yük Dengeleme ve Yük Dengeleyici Yedekliliği

6.1. Okuma/Yazma Ayrımı

HAProxy, uygulamanın tek bir adrese bağlanmasını sağlayarak istek türüne göre yönlendirme yapar: yazma istekleri birincil düğüme, okuma istekleri yedek düğüme gider. Bu ayrımın anahtarı, HAProxy'nin sabit bir düğüm listesine değil, Patroni'nin sağlık ararımına (REST, 8008 portu) bakmasıdır. HAProxy her düğüme düzenli olarak istek göndererek “sen şu an birincil misin?” (/primary) ve “yedek misin?” (/replica) sorularını sorar; yalnızca olumlu yanıt veren düğümü ilgili havuza alır. Bu sayede devralma gerçekleştiğinde HAProxy yeni birincili kendiliğinden bulur ve yazma trafiğini ona yönlendirir; uygulamanın bağlantı adresi hiç değişmez.

Çizelge 6. HAProxy port ve yönlendirme şeması

Port	Yönlendirme	Sağlık Sorgusu
5000	Yazma → yalnızca birincil	/primary yanıtı olumlu olan düğüm
5001	Okuma → yalnızca yedek	/replica yanıtı olumlu olan düğüm
7000	İzleme paneli (durum görüntüleme)	—



Şekil 17. HAProxy izleme paneli — yazma havuzunda birincil, okuma havuzunda yedek etkin

6.2. Yük Dengeleyici Katmanının Yedeklenmesi

Tek bir HAProxy örneği, ne kadar iyi yapılandırılırsa yapılandırılırsın, kendisi bir tek arıza noktası oluşturur: o sunucu çökerse veritabanı ayakta olsa bile uygulama bağlanamaz. Bu zayıflığı gidermek için ikinci bir HAProxy örneği, ikinci veritabanı düğümünde çalıştırılmıştır. Uygulama, her iki yük dengeleyiciyi de tanıyan çok adresli (multi-host) bir bağlantı dizgisi kullanır:

```
postgres://<kullanici>:<parola>@10.10.0.1:5000,10.10.0.2:5000/postgres?target_session_attrs=read-write
```

Bu dizgide istemci kütüphanesi (libpq), listedeki ilk adrese ulaşamazsa kendiliğinden ikinciye dener; “salt yazma” koşulu (target_session_attrs=read-write) ise hangi yük dengeleyici üzerinden gidilirse gidilsin her zaman gerçek birincile düşülmesini garanti eder. Yapılan sınamada, birinci HAProxy kasıtlı olarak durdurulduğunda aynı bağlantı dizgisinin ikinci yük dengeleyici üzerinden birincile ulaşmaya devam ettiği ve yazma işleminin hiç kesilmediği gözlemlenmiştir. Böylece yük dengeleyici katmanı da tek arıza noktası olmaktan çıkarılmış; sistemde hem veri hem de yönlendirme katmanı yedekli hâle getirilmiştir.

7. Elde Edilen Sonular

Tüm bileşenler, birbirinden bağımsız üç gerçek sunucu üzerinde uçtan uca sınanmış ve beklenen biçimde çalıştığı doğrulanmıştır. Yüksek erişilebilirliğin olmadığı başlangıç durumu ile kurulan hedef mimari arasındaki fark aşağıdaki çizelgede özetlenmiştir.

Çizelge 7. Önce/sonra karşılaştırması

Yetenek	HA Öncesi	HA Sonrası (Doğrulanmış)
Çoğaltma	Tek kopya, yedek yok	Yazılan kayıt yedekte anında görünür (gecikme 0)
Yedek koruması	—	Yedeğe yazma reddedilir (salt-okunur)
Otomatik devralma	Birincil ölünce hizmet biter	Çoğunlukla otomatik yeni lider seçilir
Geri katılım	—	Eski birincil, yedek olarak kayıpsız geri katılır
Yük dengeleme	Tek adres, tek düğüm	Yazma→birincil, okuma→yedek ayrımı
LB yedekliliği	Tek HAProxy = arıza noktası	İki HAProxy, istemci kendiliğinden geçer

Sonuç olarak mimaride hiçbir katman tek arıza noktası bırakmamaktadır: veritabanı düğümlerinden biri çökse Patroni devralmayı yürütür; yük dengeleyicilerden biri çökse istemci diğerine geçer; etcd düğümlerinden biri çökse çoğunluk korunur.

8. Karşılaşılan Problemler ve Çözümleri

Gerçek, dağıtık ve heterojen bir ortamda kurulum yapıldığı için, tek makinelik bir benzetimde ortaya çıkmayacak çeşitli sorunlarla karşılaşmıştır. Bu sorunların çözüm süreçleri, projenin öğretici değerinin önemli bir bölümünü oluşturmaktadır.

8.1. Güvenlik Duvarının Ağ Trafiğini Engellemesi

Sunuculardan birinde varsayılan olarak gelen istekleri reddeden güvenlik duvarı (UFW), WireGuard tüneli içindeki TCP trafiğini (etcd portları) düşürüyordu. İlginç biçimde ICMP (ping) paketleri geçerken TCP bağlantıları kurulamıyordu; bu da sorunun teşhisini güçleştiriyordu. Söz konusu durum, hem etcd düğümlerinin birbirine bağlanamamasının hem de Patroni'nin yanıltıcı hata mesajları üretmesinin asıl köküydü. Çözüm olarak güvenlik duvarına, yalnızca özel ağ arabirimi üzerinden gelen trafiğe izin veren tek bir kural eklenmiştir. Bu sorun, dağıtık sistemlerde ağ katmanının veritabanı katmanından önce doğrulanması gerektiğini açıkça göstermiştir.

8.2. etcd Küme Sürümünün Yanlış Raporlanması

Üç düğümün eşzamanlı olarak “yeni küme” modunda başlatılması durumunda, etcd küme sürümü düşük bir değerde takılı kalıyor ve Patroni bu değere bakarak artık desteklenmeyen eski bir arabirimi kullanmaya çalışıyordu. Çözüm olarak küme önce tek düğümle başlatılmış, ardından diğer iki düğüm sırayla “öğrenen (learner)” üye olarak eklenip senkronlandıktan sonra tam üyeliğe terfi ettirilmiştir. Bu kademeli yöntem, küme sürümünün doğru biçimde yükselmesini sağlamıştır.

8.3. Konteyner ve Ağ Kaynaklı Diğer Sorunlar

PostgreSQL'in ilk kurulum adımı (initdb), güvenlik gereği yönetici (root) kullanıcısıyla çalışmayı reddettiğinden, Patroni konteyneri ayrıcalıksız bir kullanıcıyla çalışacak biçimde

yeniden yapılandırılmıştır. Ayrıca AWS düğümünün ağ arabirimi için fazla büyük olan paket boyutu (MTU değeri), WireGuard tüneline paketlerin parçalanıp kaybolmasına yol açtığından 1420 değerine sabitlenmiş ve bu ayar kalıcı hâle getirilmiştir. Son olarak, başarısız kurulum denemelerinden etcd üzerinde kalan eski kayıtların kümeyi “lider bekleniyor” döngüsüne soktuğu fark edilmiş; bu kayıtlar temizlenerek kümenin temiz biçimde kurulması sağlanmıştır.

9. Sonuç ve Değerlendirme

Bu çalışmada, tek sunucu bağımlılığının tümüyle ortadan kaldırıldığı, gerçek anlamda dağıtık ve kendi kendini onaran bir veritabanı mimarisi kurulmuştur. Birincil sunucu çöktüğünde sistem, üç düğümlü bir konsensüs mekanizması sayesinde insan müdahalesi olmadan saniyeler içinde yeni bir lider seçmiş; uygulama, bağlantı adresini hiç değiştirmeden kesintisiz çalışmaya devam etmiştir. Özellikle “tanık düğüm” yaklaşımının, iki düğümlü kurulumların devralma yapamama sorununu nasıl çözdüğü uygulamalı olarak gösterilmiştir.

Projenin en önemli kazanımlarından biri, dayanıklılığın tek bir önlemle değil katmanlı bir tasarımla sağlandığının gösterilmesidir: ağ katmanı şifreli ve izole, karar katmanı çoğunluk temelli, veri katmanı çoğaltmalı ve yönlendirme katmanı yedeklidir. Ayrıca farklı işlemci mimarilerine ve farklı sağlayıcılara sahip sunucuların tek bir tutarlı küme hâlinde çalıştırılması, gerçek dünyadaki heterojen dağıtık sistemlerin de bu yöntemlerle yönetilebileceğini ortaya koymaktadır. Kurulan yapı, dersin konusu olan ağ tabanlı dağıtık sistemlerin dayanıklılık ilkesini gerçek makineler ve gerçek ağ üzerinde bütünüyle yansıtmaktadır.

KAYNAKLAR

- [1] The PostgreSQL Global Development Group, “PostgreSQL 16 Documentation”, <https://www.postgresql.org/docs/16/>
- [2] The PostgreSQL Global Development Group, “PostgreSQL Documentation: Chapter 26. Backup and Restore (Continuous Archiving and PITR)”, <https://www.postgresql.org/docs/16/continuous-archiving.html>
- [3] Crunchy Data, “pgBackRest - Reliable PostgreSQL Backup & Restore (User Guide)”, <https://pgbackrest.org/user-guide.html>
- [4] Oracle Corporation, “Oracle Database Security Guide, 23ai”, <https://docs.oracle.com/en/database/oracle/oracle-database/23/dbseg/>
- [5] Oracle Corporation, “Oracle Database PL/SQL Packages and Types Reference: DBMS_CRYPTO”, https://docs.oracle.com/en/database/oracle/oracle-database/23/arpls/DBMS_CRYPTO.html
- [6] Oracle Corporation, “Oracle Database Security Guide: Auditing (Unified Auditing)”, <https://docs.oracle.com/en/database/oracle/oracle-database/23/dbseg/introduction-to-auditing.html>
- [7] OWASP Foundation, “SQL Injection Prevention Cheat Sheet”, https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- [8] Patroni Project, “Patroni Documentation - A Template for High Availability PostgreSQL”, <https://patroni.readthedocs.io/en/latest/>
- [9] etcd Authors (Cloud Native Computing Foundation), “etcd Documentation”, <https://etcd.io/docs/>
- [10] Ongaro, D. ve Ousterhout, J., “In Search of an Understandable Consensus Algorithm (Raft), USENIX ATC 2014”, <https://raft.github.io/raft.pdf>
- [11] HAProxy Technologies, “HAProxy Documentation - Reliable, High Performance TCP/HTTP Load Balancer”, <https://docs.haproxy.org/>
- [12] WireGuard / Donenfeld, J. A., “WireGuard: Next Generation Kernel Network Tunnel”, <https://www.wireguard.com/papers/wireguard.pdf>
- [13] Docker Inc., “Docker Documentation”, <https://docs.docker.com/>